

Extending τ^3 -bench into Food Delivery

A technical and operational design for the first domain in a portfolio that compounds across two-sided marketplaces.

Food Delivery v1 in 16 weeks, ~3,200 RL-ready trajectories; the same infrastructure ships domain 2 in 8 weeks.

SITUATION

Published tau-bench-extension programs already cover domains like Manufacturing, with E-commerce, Hotel, Insurance, Digital Wallet, and Consulting in progress across the field. This design targets Food Delivery as the first of several new domains for a frontier lab consuming the data.

The brief points to v1 of tau-bench, which the repo's own README flags as deprecated and redirects to τ^3 -bench. We extend τ^3 -bench (current).

COMPLICATION

The brief asks for RL training data. Most published tau-bench extensions, including the published Manufacturing release, are **eval-shaped**: ~50 hand-crafted tasks scored 0/1 by strict DB-hash equality. **RL-shaped** data needs verifier-based dense reward and broader bucket coverage; AReaL-SEA showed the payoff, lifting Pass¹ from baseline ~50% to 75% (retail) and 98% (telecom) on tau²-bench-style tasks via per-instance verifiers.

Eight retail and airline assumptions break in food delivery (multi-party, real-time, stochastic, allergen safety, two-sided marketplace, monetary identity). Four reward-shape failures follow from those breaks.

ANSWER

Three moves deliver v1 and the playbook: (1) plug a τ^3 -shaped Food Delivery domain into Sierra's architecture; (2) layer four targeted reward upgrades that turn binary outcomes into dense RL signal; (3) extract reusable infrastructure so domain 2 lands in 8 weeks.

12 capability buckets, weekly slices from week 4, 4 FTE core plus tiered contractor pyramid (~85 active at peak), 50% of v1 infra reused on domain 2 and rising to ~65% by domain 3.

We build on **two existing chasses**, Sierra's τ^3 -bench architecture and the published RL-gym methodology, and position Food Delivery within the broader agentic benchmark landscape.

Sierra τ^3 -bench (the architecture)

Brief points to v1 ([sierra-research/tau-bench](https://github.com/sierra-research/tau-bench)); v1's README redirects to τ^3 -bench ([sierra-research/tau2-bench](https://github.com/sierra-research/tau2-bench)).

- **Domain contract.** Four Python files plus four data files plug a new domain into the eval and RL framework. Submittable to taubench.com.
- **Gymnasium RL interface** via `uv sync --extra gym`. Drops directly into RL training pipelines.
- **Compositional task generator** from τ^2 -bench. Verifiable task synthesis at scale via atomic action primitives.
- **Dual-control (Dec-POMDP)**. Both agent and user have tools. Food delivery has 4 actors; we extend to multi-actor.
- **75+ task fixes** from the SABER paper (Cuadron et al., ICLR 2026).

RL-gym methodology (from public sources)

Public framing on and the tau-bench-extension blog. We adopt the same vocabulary.

- **~50% pass-rate calibration target** for "Tau-style RL-gyms." We hit it via difficulty stratification.
- **Three-strategy QA:** human-in-the-loop review, multi-layered validation, collaborative evolution.
- **"Can your AI say no?"** framing for policy-aware agents. Bucket 7 is built around it.
- **Failure taxonomy:** tool execution, data grounding, reasoning. Drives our auto-error-identification.
- **Existing program slot.** Manufacturing live; E-commerce, Hotel, Insurance, Digital Wallet, Consulting in development. Food Delivery slots in, does not replace.

Where Food Delivery sits in the landscape. SWE-bench Verified (500 tasks, top frontier model ~93% Pass¹) tests coding agents. AgentBench (8 environments, 29 LLMs) tests general capability. Mind2Web 2 (130 tasks, 1,000+ human-hours) and WebArena (812 tasks) test browser agents. GAIA (466 tasks) tests assistant-style reasoning. OSWorld (369 tasks, humans 72% vs best model 12%) tests OS agents. Tau-bench's lane is multi-turn tool use under domain policy with a simulated user. Food Delivery sits in that lane and adds marketplace primitives the others do not exercise.

Food delivery breaks **eight retail and airline assumptions**. Each break adds a new env, tool, or reward primitive we build.

T ³ EXISTING PATTERN	FOOD DELIVERY REALITY	NEW MECHANISM WE BUILD
2-party (customer, agent)	4-party: customer, restaurant, driver, support	Multi-actor user-simulator, <code>user_tools.py</code> per role
Static DB	Real-time temporal state, order being prepped, driver moving, surge changing	Time-stepped env, <code>_env_clock_minutes</code> advances on each turn
Fully observable items / inventory	Partial observability, ETA distribution, hidden kitchen capacity	Stochastic transitions, ETA distributions in <code>track_order_eta</code>
Deterministic outcomes	Stochastic, driver reassigned, restaurant rejects, item OOS mid-prep	Equivalence-class hashing (any driver in candidate set counts as correct)
One-shot resolution	Live recovery, food cannot be re-delivered, only refunded, replaced, credited	Multi-step recovery rubric, process reward checkpoints fire mid-trajectory
Identity = lookup	Geofencing, address fuzz, dietary flags, allergens, group orders	Safety-gate hard rules (allergen violation hard-zeros episode)
Single-DB state	Two-sided marketplace, restaurant capacity AND driver supply both constrained	Marketplace mechanics: surge curves, capacity tables, payout ledger
Refunds = monetary delta	Refund × credit × replacement × payout reversal, many valid resolutions	Monetary-identity invariant , sum must equal disputed amount

These same patterns generalise to ride-share, last-mile logistics, healthcare scheduling, B2B procurement, field service. **That makes Food Delivery the right unit-1 domain.**

The architecture is small and orthogonal. A new domain plugs in via **three contracts**: typed Action and Task models, a Tool ABC, and an env that hashes DB state for reward.

`tau_bench/types.py`

```
class Action(BaseModel):
    name: str
    kwargs: Dict[str, Any]

class Task(BaseModel):
    user_id: str
    actions: List[Action] # GT replay sequence
    instruction: str # persona for user-sim
    outputs: List[str] # substring assertions
```

`tau_bench/envs/tool.py`

```
class Tool(abc.ABC):
    @staticmethod
    def invoke(*args, **kwargs) -> str:
        raise NotImplementedError

    @staticmethod
    def get_info() -> dict: # OpenAI fn-calling JSON
        raise NotImplementedError
```

`tau_bench/envs/base.py`, `calculate_reward` (excerpt)

```
data_hash = self.get_data_hash()
self.data = self.data_load_func() # fresh DB
for action in self.task.actions: # replay GT
    self.step(action)
gt_data_hash = self.get_data_hash()
reward = 1.0 if data_hash == gt_data_hash else 0.0
```

What this means for our build

Typed actions, in-place mutation. Tools mutate `data` dict in place; that mutation is what reward hashes after the trajectory ends. Our Food Delivery domain ships 24 agent tools and 12 user tools (3 personas × 4 each) following the same ABC.

Reward = strict hash equality. SHA-256 of the agent's final DB state must equal the hash after replaying ground-truth actions. Strict 0/1, no partial credit, no tolerance for stochastic outcomes, no safety constraints. Fine for eval, wrong shape for RL training.

Where our four reward upgrades attach. We extend `calculate_reward()` with: (1) canonical hashing on equivalence-class fields (handles stochastic driver assignment); (2) safety-gate predicate (hard-zeros allergen violations); (3) monetary-identity check (accepts any valid refund / credit / replacement combination summing to the disputed amount); (4) per-checkpoint process reward (dense RL signal).

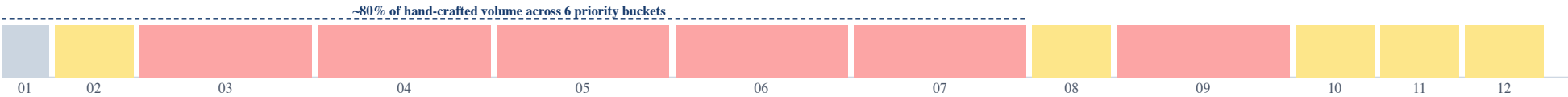
Backwards-compatible. Tasks without `reward_overrides` reduce to base τ^3 reward. Existing retail and airline leaderboards still work; food delivery and future domains layer their overrides per task.

Six of twelve buckets carry ~80% of expected reward gap. Hand-crafted depth goes there; compositional generation covers the rest.

Bucket the capability surface. Run consuming-lab model on a 50-task probe in W1. Compute Pass¹ and Pass⁴ per bucket. Concentrate hand-crafted investment where reward gap × business value is largest. Six buckets pass that filter.

01 Order placement browse, cart, ETA quote	02 Order modification add/remove, swap, special instructions	03 Cancellation & refund state × initiator × eligibility	04 Substitution & OOS stochastic event, consent, safety
05 Driver coordination re-route, contact, missing, reassign	06 Restaurant coordination prep delay, capacity, rejection	07 Compensation "Can your AI say no?", policy under pressure	08 Delivery exceptions wrong addr, damage, contamination
09 Dietary & allergy safety-critical, hard-block	10 Group / scheduled / multi-stop compositionally complex	11 Loyalty, promo, surge time-stamped pricing rules	12 Fraud, abuse, violation adversarial personas, honeypots

INVESTMENT ALLOCATION, % OF 200 HAND-CRAFTED TASKS PER BUCKET



Hypotheses falsified in W1 against the actual consuming-lab model on a 50-task probe set; tree updated by W2.

Same architecture as τ^3 retail. 30× the task volume. Multi-actor user simulation where retail had two parties.

Domain files, the τ^3 -bench shape

```
src/tau2/domains/food_delivery/
├─ data_model.py      # 11 tables, subclass of DB
├─ tools.py           # 24 agent tools (13 read + 7 write + 4 escalation)
├─ environment.py     # env factory + tasks loader
├─ user_data_model.py # dual-control state
├─ user_tools.py      # 12 tools, 3 personas × 4

data/tau2/domains/food_delivery/
├─ tasks.json         # ~3,200 tasks
├─ split_tasks.json   # base, train, dev, test, 3 curriculum
├─ db.json            # initial DB state
├─ policy.md          # ~30 pages, the wiki
```

Multi-actor user simulator (Dec-POMDP, extended)

Three personas: **customer** (4 tools: provide_info, accept and refuse_substitute, terminate), **driver** (update_location, mark_pickup, mark_delivered, report_issue), **restaurant** (mark_received, mark_prep_started, mark_oos, mark_ready). All act on shared env state. Each persona × 4 strategies (cooperative, adversarial, realistic, impatient) gives 12 simulator configurations per task.

Component sizing vs τ^3 retail

COMPONENT	FOOD DELIVERY V1	τ^3 RETAIL
Data tables	11	3
Agent tools (13 read + 7 write + 4 escalation)	24	16
User tools (3 personas × 4)	12	0
Initial DB: restaurants/drivers/customers/items	50/80/200/500	n/a
Hand-crafted seed tasks	200	~107
Compositionally generated tasks	3,000	n/a
Total tasks shipped at v1	3,200	~107
Process reward checkpoints (avg / task)	5	0
Splits (base, train, dev, test, 3 curriculum)	7	3

Volume anchors. the Manufacturing extension released 50 tasks. SWE-bench Verified ships 500. GAIA ships 466. Mind2Web 2 ships 130 at 1,000+ human-hours. MUA-RL trained against 165 tau-bench tasks (115 retail + 50 airline) and lifted retail Pass¹ to 67%. We ship 200 hand-crafted seed plus 3,000 generated to give RL training the per-bucket density (250-400 trajectories per priority bucket) that public eval sets do not.

Base τ^3 reward is binary 0/1, fine for eval, wrong for RL. Four targeted upgrades close the failure modes food delivery surfaces.

1 · EQUIVALENCE-CLASS HASHING

Stochastic outcomes don't break reward.

Driver assignment is non-deterministic; any driver in the valid candidate pool is correct. Canonical hashing on candidate-set projection succeeds where strict equality fails. **~10 LOC**, buckets 5, 6, 10.

3 · MONETARY-IDENTITY INVARIANT

refunds + credits + replacement = disputed

A refund scenario has many valid resolutions. Reward checks the identity, not one specific ledger pattern. With **hard_zero_if_above**, "Can your AI say no?" becomes trainable in one signal. **~25 LOC**, buckets 3, 7, 8, 11.

2 · SAFETY-GATE HARD RULES

Allergen violation hard-zeros the episode.

Some violations cannot be recovered. Pre-check predicate runs against the trajectory; if violated, reward = 0 unconditionally. **~30 LOC + DSL**, buckets 7, 9, 12.

4 · DENSE PROCESS REWARD

Per-checkpoint partial credit unlocks RL.

Sparse 0/1 is hard credit-assignment. Each task ships ~5 checkpoint predicates with weights. The lab trains on outcome, process, or any custom blend. **~50 LOC + DSL**, **100x signal density**.

Upgrade 2 in code, the allergen safety gate

tau2/domains/food_delivery/reward.py (excerpt)

```
def _safety_gate_passed(self) -> bool:
    gates = self.task.reward_overrides.get("safety_gate", [])
    for gate in gates:
        if gate["kind"] == "no_allergen_substitute":
            cust = self.data["customers"][self.task.customer_id]
            allergens = set(cust["allergens_hard_flag"])
            for call in self.actions:
                if call.name == "request_substitute":
                    sub = self.data["menus"][(
                        call.kwargs["restaurant_id"],
                        call.kwargs["substitute_item_id"])]
                    if set(sub["allergens"]) & allergens:
                        return False # hard-zero
    return True
```

Composed in `calculate_reward_shaped()` as `safety × (outcome + 0.3 × process) × monetary`. AReaL-SEA validated this style of verifier-based shaping: per-instance verifiers in tau²-bench drove Pass¹ from baseline to 75% retail and 98% telecom. Allergen safety, refund accounting, and policy compliance are **responsible-AI primitives built into reward**, **not bolted on**: matches the standard "data privacy, bias mitigation, model monitoring, human-in-the-loop workflows."

First integratable slice in week 4; weekly increments keep us calibrated to a moving model target.

W1,2 Scope & gap analysis — Decision tree v1 — 50-task probe — Bucket-level reward gap — Hiring funnel started	W3,4 Domain skeleton, 1st slice ships — data_model + 24 tools — policy.md v1 — 50 hand-crafted tasks — Consuming lab begins grading	W5,8 Bucket coverage, weekly slices — 3-persona user simulator — All 4 reward overrides live — 200 hand-crafted tasks — 4 review checkpoints	W9,12 Compositional + scale — Generator yield ~70% — +2,100 generated tasks — Telemetry / fraud live — Process checkpoints attached	W13,14 Reward calibration — Difficulty re-weighted to ~50% pass — LLM-judge audit (1% sample) — Curriculum splits scored — 3,000 task target hit	W15,16 Freeze, ship, retro — Final QA pass — Splits + signoff — Leaderboard submission — Domain-2 starter extracted
--	---	--	---	--	---

Column widths scale to weeks; highlighted phases ship to the consuming lab every week

FIRST SLICE

W4 / 16

~50 tasks for pipeline integration; consuming lab begins reviewing.

REVIEW CHECKPOINTS

13

Weekly slice review from W4 onward.

TASKS AT V1

3,200

200 hand-crafted + 3,000 generated.

BUCKET COVERAGE

12 / 12

All shipped by W8; densified W9 to W14.

Small core, tiered contractor pyramid. ~50% of v1 infrastructure carries to domain 2 and **climbs to ~65% by domain 3** once abstractions stabilise.

Org pyramid (peaks W9 to W12)

FTE	Program Director, Project Mgr, Domain Architect, QA & Methodology lead	core team	4
T1	Domain experts: ex-DoorDash / Uber-Eats ops PM, ex-restaurant ops director	\$120 / hr	4
T2	Senior SWE / data eng: env, generator, telemetry, grader regression	\$80 / hr	4
T3	Trajectory writers: author tasks, run rollouts, self-check	\$30 / hr	~50
T4	Reviewers: paste-detection, fraud audit, replay verification	\$15 / hr	~25

Rates anchored to public labeling-platform rate cards (\$30 to \$90/hr) and Mercor comparables (avg \$95/hr, specialised up to \$300/hr). Internal headcount stays flat; the pyramid scales horizontally per program.

What carries across domains 2..N

REUSABLE INFRASTRUCTURE	REUSE TARGET
Compositional generator framework (orchestrator + verifier DSL); per-domain atomic libraries are new	70%
User-simulator framework + telemetry	85%
Grader regression suite tooling; per-domain golden trajectories are new	80%
Behavioural fraud / paste-detection pipeline	100%
Hiring funnel templates and rate cards	90%
Two-sided-marketplace VRN pattern	if applicable
Curriculum / difficulty annotation schema	100%
Weighted average, domain 2 (8 weeks)	~50%
Weighted average, domain 3+ as abstractions stabilise	~65%

What's per-domain new work

- `data_model.py` , `tools.py` , `policy.md` , plus the per-domain atomic action library for the generator
- ~150 hand-crafted seed tasks plus new T1 vertical-specific domain experts

Eleven scale risks, each with a **named mechanism. The brief asks specifically about this layer; the highlighted three carry the largest weekly variance in \$/task and quality.**

RISK	NAMED MECHANISM
▲ Throughput cap above ~1,000 tasks	Compositional generator scales easy buckets; hand-craft only what the generator cannot cover; parallel cells with isolated DBs.
▲ Margin erosion under deadline pressure	Every step routes to cheapest viable tier. T1 at design + sampled QA only. Weekly \$/task by-bucket dashboard. Kill buckets that won't hit margin.
▲ Reward function correctness, the grader is the bottleneck	Grader regression suite of 350 trajectories: 200 known-good, 100 known-bad, 50 boundary edges. CI gate on every grader change. LLM-judge audit on 1% of new tasks.
Quality drift at scale	10% double-grading sample, golden-task probes (10/week per writer), per-writer pass-rate tracking, weekly calibration.
AI leakage / fraud	Behavioural telemetry: paste timing, mouse, keystroke cadence, honeypot tasks, group-fraud detection (flag, gather, ban as cluster).
Hiring 50 T3 writers fast	AI work-sample test (3 trajectories auto-graded), human pair-review on first 5 paid tasks, ~12% target conversion through the funnel, ~\$15K funnel cost.
Trajectory diversity	Persona × strategy matrix forced (3 × 4). Compositional generator parameterised for context randomness. Edit-distance distribution monitored weekly.
Consuming-lab researcher translation	PD owns translation. Written one-pager spec per bucket signed off W2. Weekly slice review forces specification refinement.
No room for mistakes (top labs QA via competitors)	Triple-check: deterministic grader + internal LLM-grader + 1% human audit. Audit trail per task.
Annotator burnout, domain expertise gap	Bonus tied to grader-pass-rate. T3 to T1 promotion path. T1 hires from real food-ops orgs; policy.md authored by them, not LLMs.
Dataset quality measurement	Four metrics shipped weekly: bucket-coverage entropy, edit-distance diversity, difficulty calibration (per-bucket pass-rate vs target), grader-judge agreement (Cohen's κ).

Scale ships volume. Surge ships expert review. Mercor ships sourcing. **We ship a τ^3 -shaped RL gym** with reward upgrades, a multi-actor simulator, and infrastructure that carries to the next domain.

How approaches differ on the same brief

VENDOR	DEFAULT APPROACH	GAP ON THE RL-DATA ASK
Scale	Vertical-scale annotation; major footprint in defence Gen-AI (Donovan, \$300M+ DoD).	Eval-shaped task labels and SFT pipelines, less custom env / verifier-reward work.
Surge	Smaller, higher-quality expert pool; deep human review for RLHF preference labels (Anthropic RLHF customer; ~\$1.2B ARR).	Per-task cost stays high; harder to scale via compositional generation.
Mercor	Domain-expert sourcing first; APEX-Agents 480-task benchmark across corporate-law, IB, consulting tasks (\$1B revenue, \$10B valuation).	Methodology and tooling stitched per program; less reusable infra across domains.
Invisible	Workflow-automation tooling and human-feedback ops (Cohere Command-A case study; \$2B valuation).	Less specialised on agentic env / verifier-reward design.
This proposal	τ^3 -shaped RL gym; four reward upgrades; multi-actor simulator; compositional generator; dataset-quality KPIs; infra reusable across domains.	Covers the gap above and adds the marketplace primitives.

Two original moves that compound

1 · VIRTUAL RESTAURANT NETWORK (VRN)

Persistent two-sided marketplace simulator with shared schema across vertical domains.

50 fictional restaurants, 80 drivers, 200 customers; menus, prep-time profiles, capacity rules. The schema (supplier x courier x consumer x pricing engine) is the abstraction; swap restaurants for clinics, sellers, or technicians and the same generator and simulator produce ride-share, healthcare-scheduling, marketplace, or field-service trajectories.

2 · DIFFICULTY-ANNOTATED CURRICULUM SPLITS

The consuming lab trains curriculum-RL out of the box.

Every task tagged: `difficulty` , `multi_step_depth` , `recovery_required` , `multi_party` . Adds minutes per task; differentiates from flat datasets at near-zero marginal cost.

Reliability is not the same problem as intelligence. the public frame for tau-bench. The data we build is what closes that gap; the playbook makes the next domain cheaper than this one.